

Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas



FIUSAC
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Título del Proyecto: **Estación de Control de la Nave Espacial (Star Wars)**

PONDERACIÓN: 30 pts

 **Tiempo estimado: 50 hrs**

Índice

1. MARCO FORMATIVO	3
1.1. Valor	3
1.2. Competencia(s)	3
1.3. Objetivo SMART	3
2. Resumen Ejecutivo	4
3. Enunciado del Proyecto	4
3.1 Descripción del problema a resolver	4
3.2 Alcance del proyecto	4
3.3 Entregables	5
4. Material de apoyo	7
5. Metodología	7
6. Recursos y herramientas a utilizar	8
7. Desarrollo de Habilidades Blandas	8
8. Cronograma	9
9. Rúbrica de Calificación	10
9.1 Requisitos para optar a la calificación	10
9.2 Resumen de Puntuaciones	12
9.5 Comentarios Generales	15
Detalle de la Calificación	15

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Persistencia y Análisis de Datos	Se aplica mediante el registro continuo de telemetría y eventos en MongoDB. El valor es entender cómo los datos generados por el hardware se almacenan de manera estructurada para análisis posterior (gráficas, estadísticas) y toma de decisiones.

1.2. Competencia(s)

Tipo de Competencia	
Competencia General	Capacidad de escribir y depurar código en Python para interactuar directamente con el GPIO de la Raspberry Pi, controlando múltiples sensores (digitales y analógicos) y actuadores (servomotores, motores paso a paso y ventiladores) de forma simultánea y estable.
Competencia Específica	Aplica conceptos de electronica para la construccion de prototipos IoT orientados a casos de uso del mundo real mediante placas de desarrollo como raspberry pi, sensores digitales y diseño de sistemas para la automatizacion de procesos logrando mayor eficiencia

1.3. Objetivo SMART

SMART	Definición	Objetivo redactado
Específico (¿Qué?)	El objetivo es concreto y tangible.	Los estudiantes serán capaces de diseñar, implementar e integrar múltiples subsistemas de hardware (Raspberry Pi, sensores y actuadores) con una arquitectura Full-Stack (MQTT, Node.js, React, MongoDB).
Medible (¿Cuánto?)	El objetivo tiene una medida objetiva de éxito.	Lograr que el 100% de los 7 subsistemas obligatorios (Detección de Incendios, Torreta, Camuflaje, etc.) operen de manera autónoma y controlable, demostrando la funcionalidad en tiempo real.
Alcanzable (¿Cómo?)	El objetivo debe ser posible con los recursos disponibles.	Utilizando únicamente los componentes electrónicos y las herramientas de desarrollo especificadas en los requerimientos técnicos.
Realista (¿Para qué?)	El objetivo contribuye a metas más amplias.	Con el fin de fortalecer las habilidades prácticas en IoT, arquitecturas de comunicación asíncrona (MQTT) y desarrollo de interfaces de gestión (Dashboard), preparándolos para proyectos de sistemas embebidos avanzados.

A Tiempo (¿Cuándo?)	El objetivo tiene fecha límite o mejor aún un cronograma de hitos de progreso	Completar la integración y presentación del proyecto en un plazo estimado
----------------------------	---	---

2. Resumen Ejecutivo

Este proyecto propone el diseño e implementación de una Estación de Control de Nave Espacial basada en Raspberry Pi para simular los sistemas de monitoreo y defensa ante amenazas externas (como incendios o meteoritos) en una nave de misiones de larga duración. El problema abordado es la necesidad de automatizar la detección de amenazas, gestionar el estado ambiental y permitir el control remoto y local de la nave. La solución implica la integración de múltiples sensores (gas, proximidad, color), actuadores (servomotores, ventiladores, LEDs), un dashboard web (React/Node.js) y una sala de control física. Todos los subsistemas estarán interconectados y comunicados mediante el protocolo MQTT y almacenarán la telemetría en MongoDB. El proyecto permite a los estudiantes aplicar y demostrar competencias avanzadas en IoT, programación de hardware (Python), desarrollo web Full-Stack, bases de datos NoSQL y protocolos de comunicación.

3. Enunciado del Proyecto

3.1 Descripción del problema a resolver

En las misiones espaciales de larga duración, las naves requieren sistemas automatizados de monitoreo y control que permitan detectar amenazas externas, mantener condiciones ambientales óptimas y responder ante emergencias de forma inmediata. El objetivo de este proyecto es diseñar e implementar una Estación de Control de Nave Espacial completamente funcional que integre múltiples sistemas de sensores, actuadores y una interfaz de usuario para la gestión remota y local de todos los subsistemas de la nave.

La estación debe ser capaz de:

- Detectar y responder automáticamente ante amenazas (incendios, meteoritos)
- Activar sistemas de defensa y camuflaje
- Permitir control manual desde una sala de control física
- Proporcionar monitoreo en tiempo real mediante un dashboard web
- Registrar todos los eventos y telemetría para análisis posterior

3.2 Alcance del proyecto

Alcance Obligatorio:

El proyecto debe implementar los siguientes subsistemas integrados:

1. SISTEMA DE DETECCIÓN DE INCENDIOS

- **Sensor:** MQ-2 o MQ-135 (detección de gas/humo)
- **Funcionalidad:**

- o Monitoreo continuo de niveles de gas en el ambiente
- o Activación automática de ventiladores cuando se detecten niveles peligrosos
- o Alerta visual (LED rojo parpadeante) y registro del evento
- o Envío de alerta crítica al dashboard
- o Estado visible en LCD de la sala de control

2. SISTEMA DE DETECCIÓN DE METEORITOS

- **Sensor:** HC-SR04 (sensor ultrasónico de proximidad)
- **Funcionalidad:**
 - o Detección de objetos a diferentes distancias (3 niveles: lejano, cercano, crítico)
 - o Activación de buzzers con patrones diferentes según la distancia:
 - **Lejano (>50cm):** 1 beep cada 2 segundos
 - **Cercano (20-50cm):** 3 beeps rápidos cada segundo
 - **Crítico (<20cm):** Beep continuo
 - o Visualización de distancia en tiempo real en dashboard y LCD
 - o Registro de cada detección con timestamp en MongoDB

3. SISTEMA DE CAMUFLAJE CROMÁTICO

- **Sensor:** TCS3200 o TCS34725 (sensor de color RGB)
- **Funcionalidad:**
 - o Detección de tres colores principales en secuencia (ejemplo: Rojo, Amarillo, Azul)
 - o Cuando se detecta la secuencia correcta:
 - TODAS las LED RGB alrededor de la nave replican los colores detectados (mínimo 5 LED RGB)
 - Se activa el "Modo Camuflaje" visible en dashboard
 - Se registra evento de camuflaje activado/desactivado

4. SISTEMA DE TORRETA DE DEFENSA

- **Actuador:** Motor Paso a Paso (28BYJ-48 con ULN2003 o similar)
- **Funcionalidad:**
 - o **Control desde Dashboard Web:**
 - Slider para rotar la torreta de 0° a 360°
 - Posición HOME (0°)
 - o **Control desde Sala de Control Física:**
 - Botones para girar torreta (uno para girar izquierda y otro para girar derecha)
 - Botón de disparo (activa LED láser en la punta y hace sonar un buzzer simulando el sonido de un disparo)
 - o **Comportamiento:**
 - LED en la punta de la torreta (cuando dispara se enciende la LED)

5. SALA DE CONTROL FÍSICA (INTERFAZ LOCAL)

- **Componentes:**
 - o LCD 16x2 o 20x4 para mostrar estados del sistema
 - o Mínimo 6 botones físicos:

1. **Abrir/Cerrar Puertas** (controla servomotor de compuerta)
 2. **Disparar Torreta**
 3. **Emergencia** (detiene todos los motores y activa alerta)
- o Panel de LEDs indicadores:
 - LED Verde: Sistema operativo
 - LED Amarillo: Alerta de meteorito
 - LED Rojo: Alerta de incendio
 - LED Azul: Modo camuflaje activo
 - **Funcionalidad del LCD:**
 - o Pantalla rotativa que muestra cada 3 segundos:
 - Temperatura y humedad ambiental
 - Distancia del meteorito más cercano
 - Estado de la torreta (ángulo)
 - Mensajes enviados desde el dashboard
 - o Modo alerta: muestra emergencias activas en pantalla fija
 - **Panel de Mensajes:**
 - o Debe poder recibir y mostrar mensajes de texto cortos (máx. 64 caracteres) enviados desde el dashboard
 - o Los mensajes persisten en pantalla 5 segundos
 - o Se guarda historial de mensajes en MongoDB

6. SISTEMA DE COMPUERTAS

- **Actuadores:** 1 Servomotor (SG90 o similar)
- **Funcionalidad:**
 - o **Servo - Escotilla Principal:**
 - Control exclusivo desde sala de control (botón físico)
 - Control desde dashboard (botón virtual)

7. DASHBOARD WEB (Interfaz Remota)

- **Tecnologías:**

A discreción del estudiante

Recomendadas: React + Node.js

MongoDB(obligatoria)
- **Características obligatorias:**
 - o **Panel Principal:**
 - Estado general de la nave (operativa, alerta, emergencia)
 - Visualización en tiempo real de TODOS los sensores
 - Controles para TODOS los actuadores
 - o **Sección de Sensores:**
 - Gráficas de temperatura, humedad (últimas 24 horas)
 - Indicador visual de proximidad de meteorito
 - Estado de sensor de gas (nivel actual y umbral)
 - Últimos colores detectados por sensor RGB

- o **Sección de Control:**
 - Control de torreta (slider de ángulo + botones)
 - Botones de compuertas
 - Botón de modo camuflaje manual
 - Campo de texto para enviar mensajes a la sala de control
- o **Sección de Estadísticas:**
 - Total de disparos realizados
 - Tiempo total en modo camuflaje
 - Número de alertas de gas/meteoritos

8. COMUNICACIÓN Y ALMACENAMIENTO

- **Protocolo MQTT:**
 - o Topics (sugeridos) organizados jerárquicamente:
 - nave/sensores/gas
 - nave/sensores/proximidad
 - nave/sensores/color
 - nave/sensores/ambiente
 - nave/actuadores/torreta
 - nave/actuadores/compuertas
 - nave/control/mensajes
 - nave/alertas/criticas
 - o QoS 1 para comandos críticos
 - o QoS 0 para telemetría regular
- **Base de Datos MongoDB:** (colecciones sugeridas)
 - o Colección `sensor_readings`: telemetría con timestamp
 - o Colección `events`: eventos del sistema
 - o Colección `commands`: comandos ejecutados
 - o Colección `messages`: mensajes enviados a sala de control
 - o Índices por fecha para búsquedas rápidas
- **API REST:**
 - o Endpoints para lectura de datos históricos
 - o Endpoints para envío de comandos
 - o Endpoints para estadísticas agregadas

Alcance opcional:

SISTEMA DE MONITOREO AMBIENTAL

- **Sensores:**
 - o DHT11 o DHT22 (temperatura y humedad ambiental)
 - o Sensor de humedad de suelo (simulando "zona de cultivo")
- **Funcionalidad:**
 - o Lectura continua cada 5 segundos
 - o Alertas si temperatura $>35^{\circ}\text{C}$ o $<10^{\circ}\text{C}$
 - o Alertas si humedad $<20\%$ o $>80\%$
 - o Gráficas en tiempo real en dashboard
 - o Almacenamiento de datos históricos

4.3 Requerimientos técnicos

Hardware:

- 1 Raspberry Pi 3/4 (con Raspberry Pi OS)
- 1 Sensor de Gas MQ-2 o MQ-135
- 1 Sensor Ultrasónico HC-SR04
- 1 Sensor de Color TCS3200/TCS34725
- 1 Sensor DHT11/DHT22
- 1 Sensor de Humedad de Suelo
- 1 Motor Paso a Paso 28BYJ-48 + Driver ULN2003
- 1 Servomotor SG90/MG90S
- 2 Ventiladores DC 5V
- 1-2 Buzzers activos
- Mínimo 8 LED RGB (o 1 tira NeoPixel WS2812B)
- 1 LCD 16x2 o 20x4 con módulo I2C
- Mínimo 6 botones pulsadores
- Resistencias, cables, protoboard/PCB
- Fuente de alimentación externa 5V/2-3A

Software:

- **Backend:**
 - o Python 3.8+ (control de hardware) (opcional)
 - o Node.js 16+ con Express (API REST) (opcional)
 - o Broker MQTT(obligatorio)
 - o MongoDB 5.0+(obligatorio)
- **Frontend:**
 - o React 18+(opcional)
 - o Bibliotecas de gráficas (Chart.js o Recharts) (opcional)
- **Librerías Python:**
 - o RPi.GPIO o gpiozero
 - o paho-mqtt
 - o pymongo
 - o Adafruit_DHT
 - o adafruit-circuitpython-tcs34725 (para sensor color)

3.3 Entregables

Ejemplo para proyectos de desarrollo:

Tipo	Descripción
Prototipo Físico	Nave espacial montada con todos los componentes integrados y funcionales. Debe incluir estructura física que simule la nave.
Código Fuente	Repositorio en GitHub con el código completo del proyecto, con estructura clara y comentarios.
Documentación Técnica	Informe en PDF que incluya el Diagrama de Arquitectura del Sistema (hardware, MQTT topics, stack tecnológico, Diagrama de Flujo de la lógica de respuesta automática.
Manual de Usuario	Documento que explica cómo usar el sistema desarrollado, incluyendo capturas de pantalla, pasos detallados y resolución de problemas comunes.
Diagrama en proteus	Un diagrama en proteus que refleje como esta funcionando el proyecto físicamente

4. Material de apoyo

<https://www.hivemq.com/>

<https://www.mongodb.com/es>

https://github.com/Josmalopez77/ACYE1_2S_2025

5. Metodología

Se utilizará una metodología por fases adaptable, enfocada en el desarrollo de sistemas embebidos y aplicaciones Full-Stack:

1. **Fase 1: Investigación y Diseño (Semanas 1-2):**
 - Investigación de la interconexión de sensores/actuadores con Raspberry Pi (GPIO, I2C, etc.).
 - Diseño de la Arquitectura Lógica: Creación de diagramas de flujo y esquemas de la arquitectura (hardware, MQTT Topics, Backend, Frontend).
2. **Fase 2: Desarrollo del Core y Hardware (Semanas 2-3):**
 - Implementación del Broker MQTT y la base de datos MongoDB.
 - Desarrollo de los scripts en Python (Sistema Operativo) para la lectura de todos los sensores y el control de los actuadores, incluyendo la lógica de respuesta automática (detección de gas, patrones de meteoritos).
3. **Fase 3: Desarrollo de la Interfaz (Semana 3-4):**
 - Desarrollo de la API REST (Node.js) para la persistencia de datos y la gestión de comandos.
 - Desarrollo del Dashboard Web (React), implementando la visualización en tiempo real (Socket.io) y los controles remotos de los actuadores (torreta, compuertas).
4. **Fase 4: Integración y Pruebas (Semanas 4-5):**
 - Integración final de la Sala de Control física (botones y LCD) con el sistema Core y el Dashboard Web.
 - Realización de pruebas funcionales para verificar el cumplimiento de los 7 subsistemas obligatorios y la correcta comunicación de datos (telemetría y comandos) en toda la arquitectura (RPi -> MQTT -> BBDD -> Dashboard).

6. Recursos y herramientas a utilizar

Categoría	Elementos
Hardware	1 Raspberry Pi 3/4 (con Raspberry Pi OS)
	1 Sensor de Gas MQ-2 o MQ-135
	1 Sensor Ultrasónico HC-SR04
	1 Sensor de Color TCS3200/TCS34725
	1 Sensor DHT11/DHT22 (Ambiente)
	1 Sensor de Humedad de Suelo (Opcional)
	1 Motor Paso a Paso 28BYJ-48 + Driver ULN2003 (Torreta)
	1 Servomotor SG90/MG90S (Compuertas)
	2 Ventiladores DC 5V, 2 Buzzers activos, Tiras o mínimo 8 LED RGB
	1 LCD 16x2 o 20x4 con módulo I2C, Mínimo 6 botones pulsadores
Software Core	Python 3.8+ (Control de hardware), Librerías: RPi.GPIO o gpiozero, paho-mqtt, pymongo, Adafruit_DHT, adafruit-circuitpython-tcs34725 (para sensor color)
Software Full-Stack	Node.js 16+ con Express (API REST), React 18+ (Frontend), Broker MQTT, MongoDB 5.0+ (Base de Datos)

7. Desarrollo de Habilidades Blandas

El equipo debe asignar roles de responsabilidad exclusivos para cada subsistema clave y tecnología. Se sugiere la siguiente división:

Rol Sugerido	Responsabilidad Central	Subsistemas / Tecnologías Asociadas
Arquitecto de Hardware (Líder RPI)	Lidera la implementación de los scripts Python y la lógica de control principal. Garantiza la estabilidad del hardware.	Sistemas de Detección (Incendios, Meteoritos Sala de Control Física (LCD, Botones) Configuración del Broker MQTT y RPI
Ingeniero de Actuadores y Mecánica	Responsable de la correcta integración física de motores y actuadores.	Sistema de Torreta de Defensa (Motor Paso a Paso) Sistema de Puertas (Servomotor) Sistema de Camuflaje (LEDS y Sensor de Color)
Desarrollador Backend (API)	Diseña, implementa y mantiene la API REST y la conexión con la base de datos.	Node.js / Express (API REST) Integración con MongoDB Gestión de tópicos MQTT de comandos y persistencia
Desarrollador Frontend (Dashboard)	Diseña y desarrolla la interfaz de usuario web (Dashboard).	React (Componentes, UX/UI) Visualización de datos en tiempo real (Gráficas, Controles) Comunicación con la API y Socket.io
Especialista en QA y Documentación	Lidera las pruebas de integración (end-to-end) y es responsable de la documentación formal del proyecto.	Elaboración del Manual de Usuario y Diagramas de Arquitectura Pruebas de funcionalidad de todos los 7 subsistemas Gestión del repositorio Git (integración de ramas)

8. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de Proyecto	14/02	14/02
Elaboración	14/02	20/03
Calificación	21/03	21/03

9. Rúbrica de Calificación

9.1 Requisitos para optar a la calificación

Tema	Descripción	Cumple (Sí/No)
Arquitectura de Comunicación	El Broker MQTT debe estar operativo y el flujo de datos (sensores, comandos) debe ser funcional en ambas direcciones (RPi a Dashboard y Dashboard a RPi).	
Persistencia de Datos	La telemetría de TODOS los sensores y los eventos críticos (incendio, camuflaje, disparo) deben registrarse correctamente en la base de datos MongoDB.	
Lógica de Incendios	El Sensor de Gas debe activar la lógica de respuesta automática: activación de ventiladores y alerta visual/auditiva, superando el umbral de gas.	
Alarma de Meteoritos	El Sensor Ultrasónico debe detectar las tres distancias y activar los patrones de <i>buzzer</i> diferenciados para cada nivel (lejano, cercano, crítico).	
Control de Torreta Bidireccional	El motor paso a paso debe ser controlable desde el Dashboard (slider) y los botones de Extender/Retraer/Disparar deben funcionar correctamente desde la Sala de Control Física.	
Camuflaje Cromático	El Sensor de Color debe detectar la secuencia de colores y, al lograrlo, las tiras LED RGB deben replicar los colores y la torreta debe retraerse automáticamente.	

Tema	Descripción	Cumple (Sí/No)
Panel de Control Físico (LCD)	La pantalla LCD debe mostrar información rotativa (temperatura, distancia) y debe ser capaz de recibir y mostrar mensajes de texto enviados desde el Dashboard.	
Dashboard Web (Funcionalidad Mínima)	La interfaz React debe mostrar los valores de los sensores en tiempo real y debe permitir la interacción con los controles de los actuadores (Torreta, Compuertas).	
Entregables de Documentación	El repositorio de código debe incluir las tres secciones (Python, Node.js, React) y la documentación obligatoria (Arquitectura y Manual) debe ser entregada en el formato solicitado.	

9.2 Resumen de Puntuaciones

Computación – Organización Computacional

No.	Criterio de Evaluación	Punteo Máximo	Nivel Satisfactorio (100%–61%)	Nivel Necesita Mejorar (60%–0%)	Rango de Punteo	Punteo Obtenido
1	HABILIDADES	10 ▾				
1.1	Documentación Técnica	5 ▾	Manual técnico, diagramas y arquitectura coherentes con el sistema implementado	Documentación incompleta o inconsistente	5–3 / 2–0	
1.2	Presentación y Defensa	5 ▾	Explica el sistema con dominio técnico y claridad	Explicación débil o desconocimiento del sistema	5–3 / 2–0	

	Sub-Total Habilidades	10 ▾				
2	CONOCIMIENTOS	90 ▾				
2.1	Sistema detección incendios	4 ▾	Lectura y respuesta automática correctas	Fallos de detección o respuesta	4-2 / 1-0	
2.2	Sistema detección meteoritos	4 ▾	Detección por distancia y alertas funcionales	Funcionamiento incorrecto	4-2 / 1-0	
2.3	Sistema camuflaje cromático	4 ▾	Secuencia y activación de LEDs correctas	Secuencia incorrecta	4-2 / 1-0	
2.4	Sistema torreta defensa	4 ▾	Control y disparo funcional	Control incorrecto	4-2 / 1-0	
2.5	Sistema compuertas	4 ▾	Servo responde correctamente	Fallos de control	4-2 / 1-0	
2.6	LCD y botones físicos	5 ▾	Información y controles funcionales	Visualización incorrecta	5-3 / 2-0	
2.7	Panel de LEDs indicadores	5 ▾	Estados representados correctamente	Estados inconsistentes	5-3 / 2-0	
2.8	API REST funcional	7 ▾	Endpoints correctos y estables	API incompleta o fallida	7-4 / 3-0	
2.9	Integración MongoDB	7 ▾	Telemetría y eventos almacenados correctamente	Persistencia parcial	7-4 / 3-0	
2.10	Arquitectura MQTT	6 ▾	Comunicación correcta por tópicos	Comunicación deficiente	6-3 / 2-0	

2.11	Visualización sensores	5 ▾	Datos en tiempo real correctos	Visualización incompleta	5-3 / 2-0	
2.12	Control actuadores	5 ▾	Control remoto funcional	Controles fallan	5-3 / 2-0	
2.13	Estadísticas dashboard	5 ▾	Métricas correctas	Métricas faltantes	5-3 / 2-0	
2.14	UX/UI general	5 ▾	Interfaz clara e intuitiva	Diseño confuso	5-3 / 2-0	
2.15	Comunicación full-stack	10 ▾	Flujo completo sin fallos	Integración parcial	10-6 / 5-0	
2.16	Funcionamiento global	10 ▾	Subsistemas operan coordinadamente	Fallos generales	10-6 / 5-0	
	Sub-Total Conocimientos	90 ▾				
	TOTAL GENERAL	100 ▾				

La calificación debe incluir ambas áreas conocimientos y habilidades.

9.5 Comentarios Generales
